PROJECT REPORT

Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID	SWTID-2026-6390
Team Size	4
Team Members	Pooja Padma R — Team Leader Anisha R — Team Member Santhalakshmi N — Team Member Gopika S — Team Member
Institution / Department	GCT, COIMBATORE — B.Tech INFORMATION TECHNOLOGY
Guide / Mentor	-
Academic Year	2026-2027

CONTENTS

1. INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

2.3 Brainstorming

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

3.2 Solution Requirement

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN

4.1 Problem Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

7. RESULTS

7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

1. INTRODUCTION

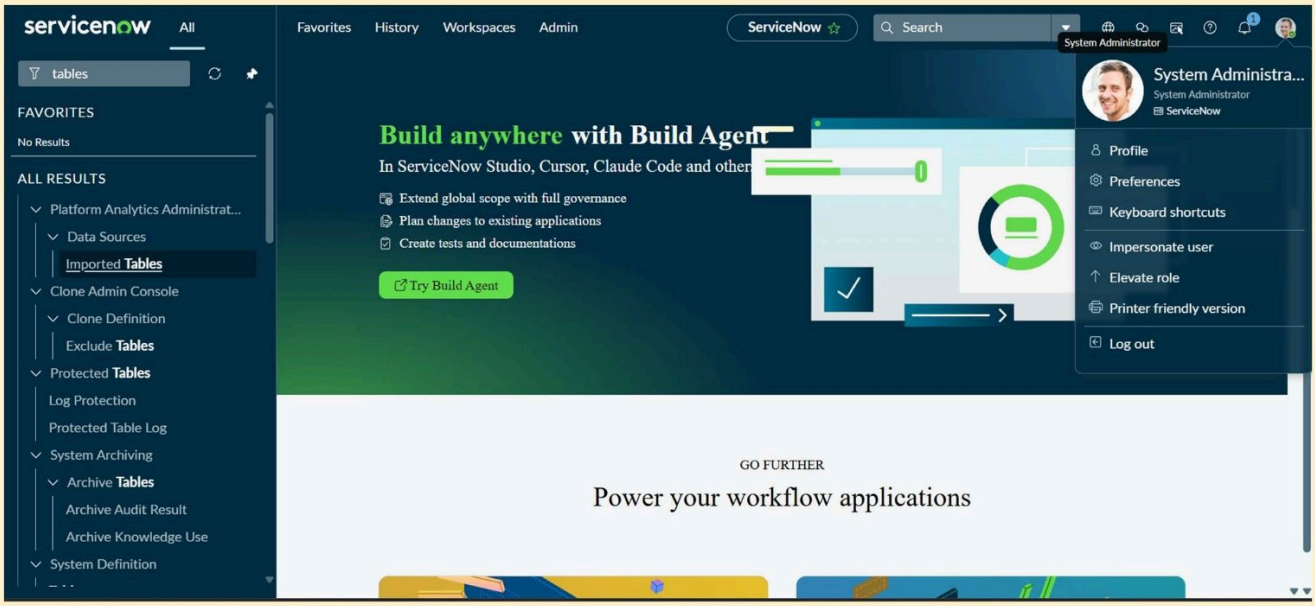
1.1 Project Overview

The project implements Script-Controlled Access Control Lists (ACLs) in ServiceNow to restrict record-level access based on user roles and the Branch field. The project uses a custom Institution Details table named u_institution_details and demonstrates separate controls for Read, Create, Write and Delete operations.

For the Read operation, the project uses the bb1 role, a Branch = EEE data condition, and an advanced script that explicitly allows administrators and bb1 users while denying others. Create, Write and Delete operations are controlled by bb2, bb3 and bb4 respectively.

1.2 Purpose

The purpose is to demonstrate how ServiceNow ACLs can enforce record-level security and control access to sensitive records. The project also demonstrates how roles and record field values can be evaluated together when making an access decision.



2. IDEATION PHASE

2.1 Problem Statement

A ServiceNow application may contain records that should not be visible or modifiable by every user. The project addresses this requirement by implementing role-based and field-based record access controls. In the demonstrated scenario, EEE branch records are protected so that users with the required role can access the permitted records, while administrators retain full Read access.

2.2 Empathy Map Canvas

An empathy map captures knowledge about a user's behaviours and attitudes and helps the team consider the user's goals and challenges. The primary perspective here is the EEE branch user who needs access to Institution Details records while unauthorized branch access is prevented

HEARS • Security rules and role requirements • Guidance that access depends on permissions	**THINKS & FEELS** **• Wants access to relevant EEE records** **• Wants confidence that data is protected** **• May worry when access is unexpectedly denied**	SEES • Institution Details list • EEE, ECE and CSE branch values • Different actions based on permissions
HEARS • Administrators explain bb1–bb4 roles • Security expectations and restrictions	**SAYS & DOES** **• Opens the Institution Details list** **• Attempts permitted operations** **• Reports unavailable records/actions** **• Uses the system according to assigned role**	SEES • Relevant EEE records • New/Edit/Delete actions when permitted • No records when READ access is denied
PAIN • Required record may be unavailable • Unexpected permission denial • Confusion about unavailable operations • Risk of seeing data outside the user's scope	**USER / CONTEXT** **EEE BRANCH USER** **Role: bb1 for READ access** **Institution Details records** **Branch-focused access**	GAIN • Relevant EEE records accessible • Other branch records protected • Clear operation permissions • Greater confidence in security

2.3 Brainstorming

Brainstorming provides a free and open environment that encourages team participation in creative problem solving. For this project, the brainstorming focuses on making ServiceNow record access secure, understandable, testable and maintainable.

The source project restricts EEE branch records through a script-controlled READ ACL, while administrators retain full access; CREATE, WRITE and DELETE are controlled through separate roles.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The project workflow can be represented as a journey from administrator configuration to user access verification. The administrator creates users and roles, configures the custom table and records, and creates ACL rules. The user then requests access to records, and ServiceNow evaluates the applicable ACL before allowing or denying the operation.

Stage	Journey Step	User / Admin Action	Expected Outcome
1	Administrator setup	Create EEE User and roles bb1–bb4	User and roles are available
2	Table setup	Create u_institution_details and records	Records contain ECE, EEE and CSE values
3	Security setup	Configure Read/Create/Write/Delete ACLs	ACL rules are active
4	User access	User opens protected records	ServiceNow evaluates access
5	Verification	Test with different role combinations	Expected permissions are observed

3.2 Solution Requirement

ID	Requirement	Description	Priority
REQ-01	User creation	Administrator can create the test user EEE User.	High
REQ-02	Role creation	Administrator can create bb1, bb2, bb3 and bb4.	High
REQ-03	Role assignment	Administrator can assign the four roles to EEE User.	High
REQ-04	Table creation	Administrator can create u_institution_details.	High
REQ-05	Record creation	Records can be created with ECE, EEE and CSE Branch values.	High
REQ-06	Read ACL	Read ACL uses bb1, Branch = EEE and the supplied script.	High
REQ-07	Create ACL	bb1 + bb2 user can access New and create a record.	High
REQ-08	Write ACL	bb1 + bb2 + bb3 user can view and edit EEE records.	High
REQ-09	Delete ACL	User with all four roles can view, create, edit and delete EEE records.	High

3.3 Data Flow Diagram

The DFD below represents the actual project workflow: the administrator configures users, roles, tables and ACL rules; the EEE user requests access; records and ACL rules are supplied to the ServiceNow ACL evaluation; and an ACL operation decision is produced.

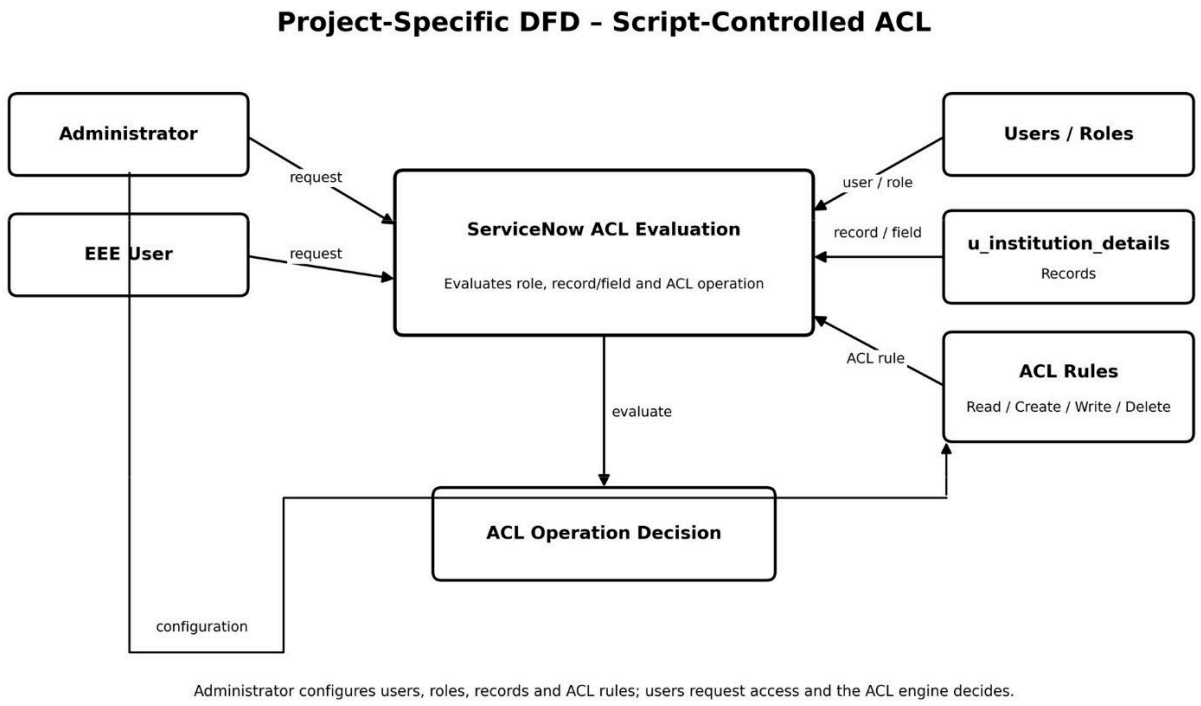


Figure 3.1: Clean project-specific DFD – Script-Controlled ACL.

3.4 Technology Stack

Component	Technology
User Interface	ServiceNow Platform UI
Administration / Configuration	ServiceNow Administration / Configuration
Read ACL Logic	ServiceNow ACL + GlideSystem (gs) script
Create / Write / Delete	ServiceNow Access Control List
Data Store	ServiceNow Platform Data Store
Infrastructure	ServiceNow Instance
External API	Not specified in supplied resource
Machine Learning Model	Not specified in supplied resource

4. PROJECT DESIGN

4.1 Problem Solution Fit

The solution fits the identified access-control requirement by applying ACLs directly to the custom table. The Read ACL combines the required role, the Branch = EEE condition and the advanced script. Separate ACLs then control Create, Write and Delete operations.

4.2 Proposed Solution

- Create a test user named EEE User.

- Create roles bb1, bb2, bb3 and bb4 and assign them as required.
- Create the custom u_institution_details table and populate records with different Branch values.
- Configure the advanced Read ACL with the bb1 role and Branch = EEE condition.
- Use separate Create, Write and Delete ACLs requiring bb2, bb3 and bb4 respectively.
- Verify access by impersonating users with different role combinations and by testing administrator access.

4.3 Solution Architecture

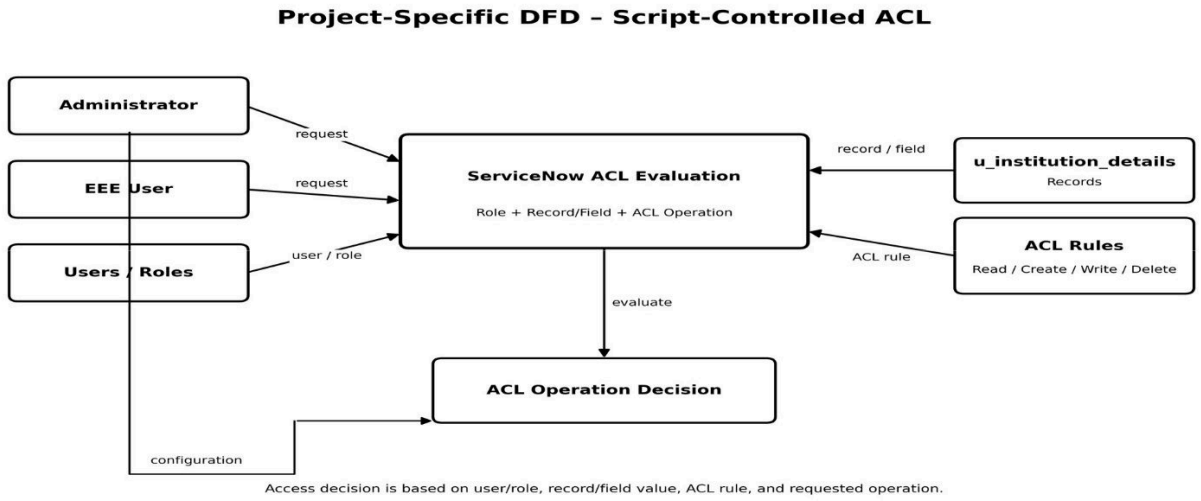
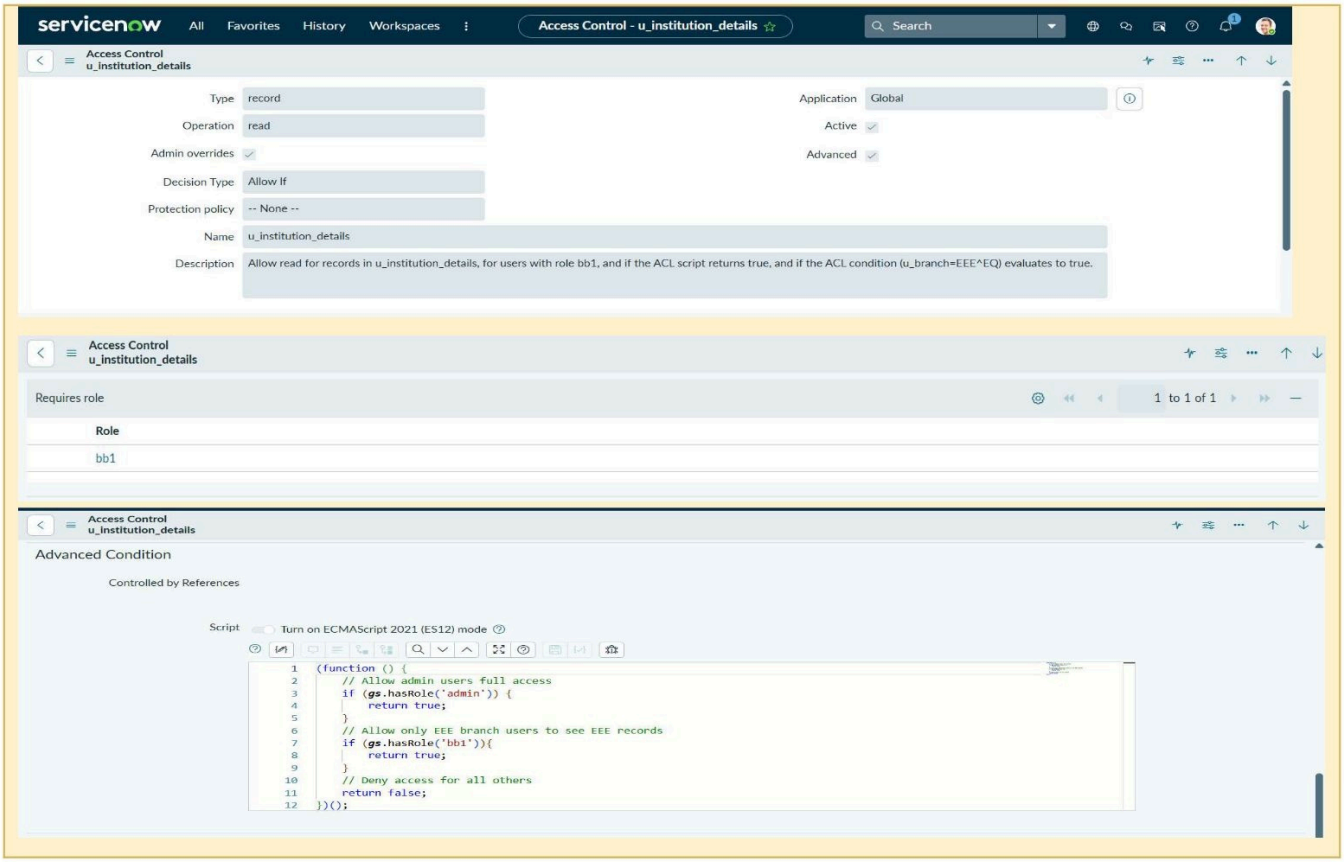


Figure 4.1: Solution architecture – ServiceNow Script-Controlled ACL.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Release	Work Package	Planned Activities
Sprint-1	Users, roles and table	EEE User; bb1–bb4; u_institution_details; fields and sample records
Sprint-2	Read and Create access	Read ACL verification; Create ACL and New-button verification
Sprint-3	Write and Delete access	Edit verification; Delete verification; final role-combination test

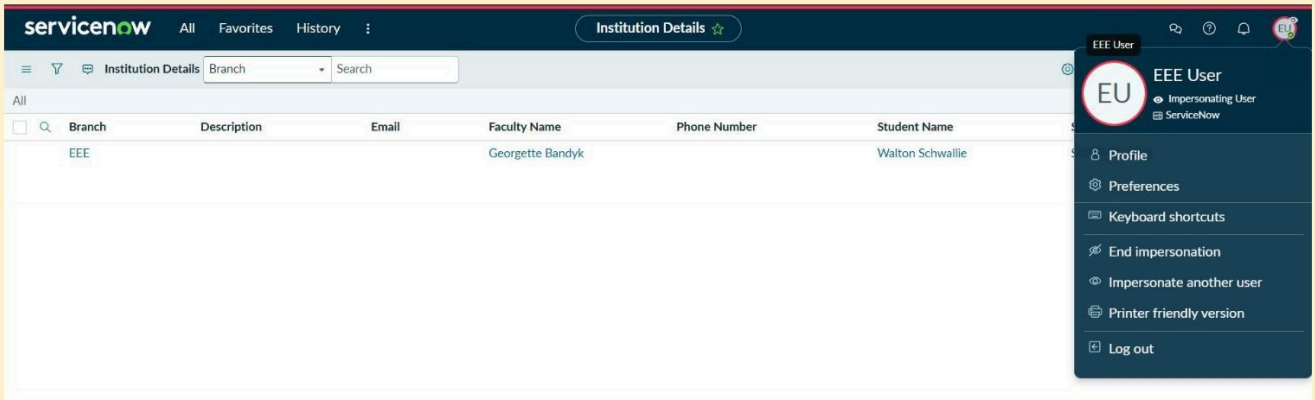
6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

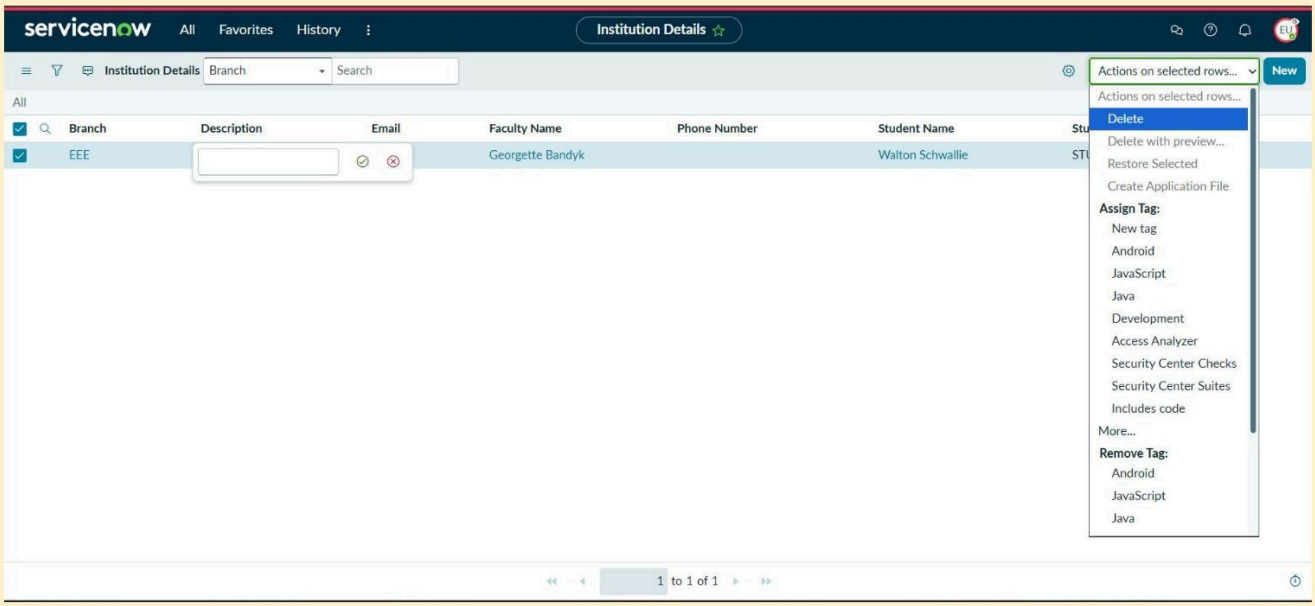
The supplied project resource defines functional acceptance criteria but does not specify request-rate, response-time, cache, CDN or load-testing requirements. Therefore, a quantitative performance result is not claimed in this report.

Test ID	User / Roles	Test Action	Expected Result	Actual Result
TC-01	EEE User with bb1	Open protected table	EEE branch records visible	Pass
TC-02	User without bb1	Open protected table	Protected records not visible	Pass
TC-03	Administrator	Open table	All records visible regardless of Branch	Pass
TC-04	bb1 + bb2	Open protected table	New button available; can create	Pass
TC-05	bb1 + bb2 + bb3	Open EEE record	Can view and edit; New available	Pass
TC-06	bb1 + bb2 + bb3 + bb4	Use record actions	Can view, create, edit and delete EEE records	Pass

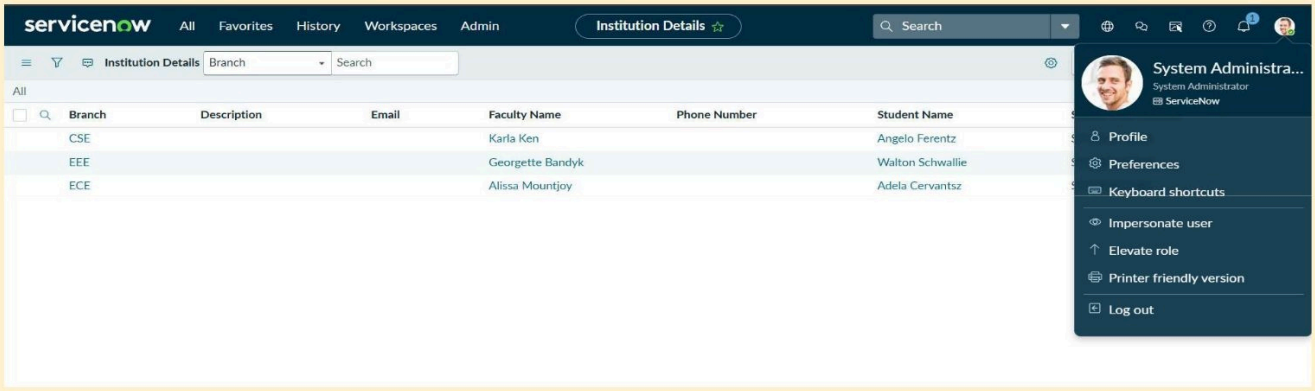
EEE user:



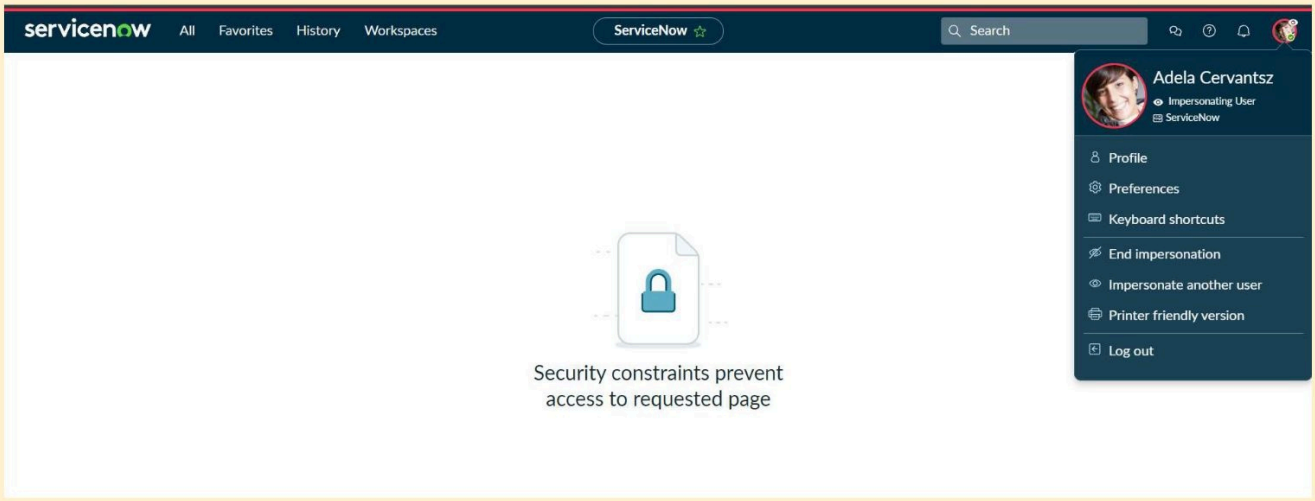
EEE user can view,modify,create and delete EEE records:



Admin has full access



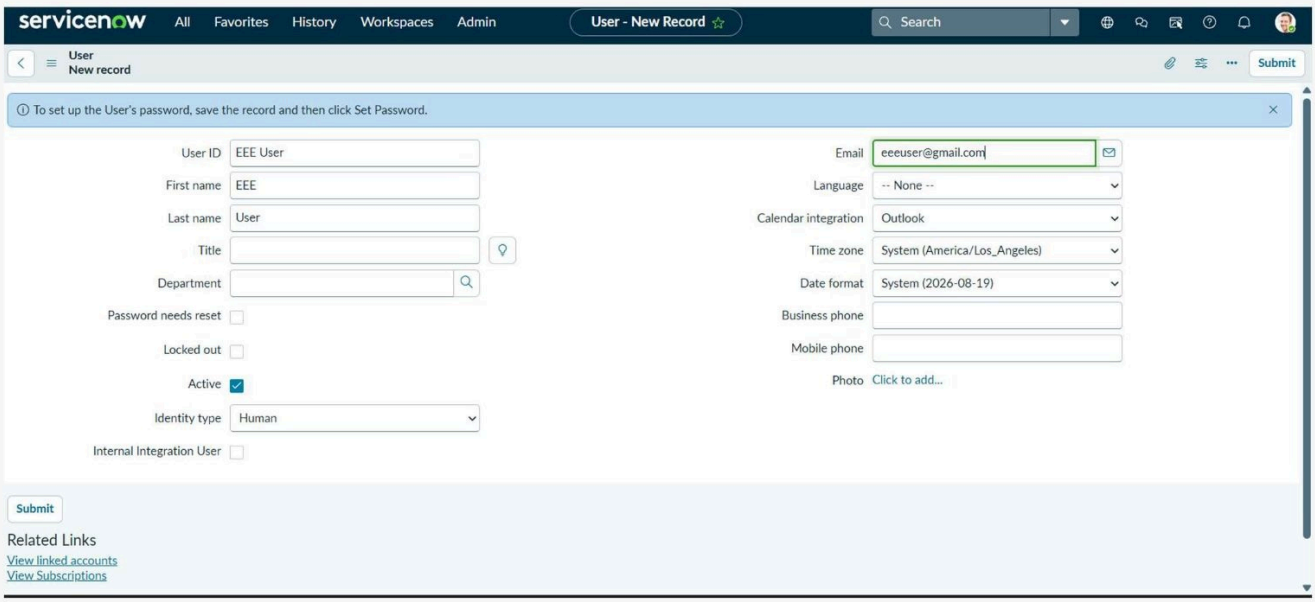
Unauthorised person can’t access:



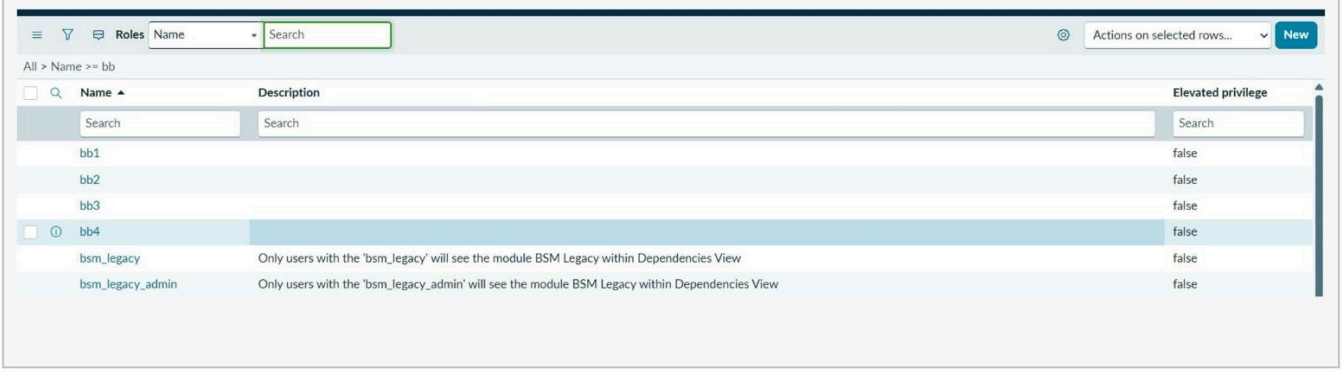
7. RESULTS

7.1 Output Screenshots

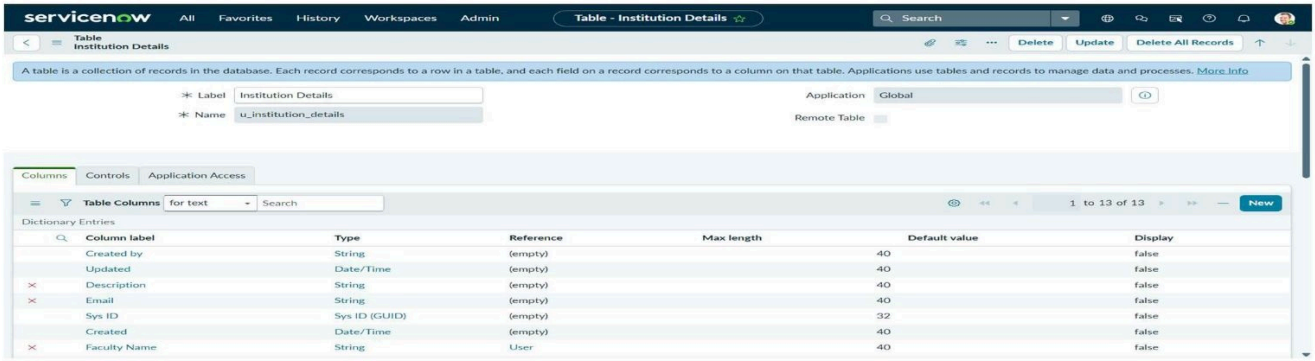
USER CREATION:



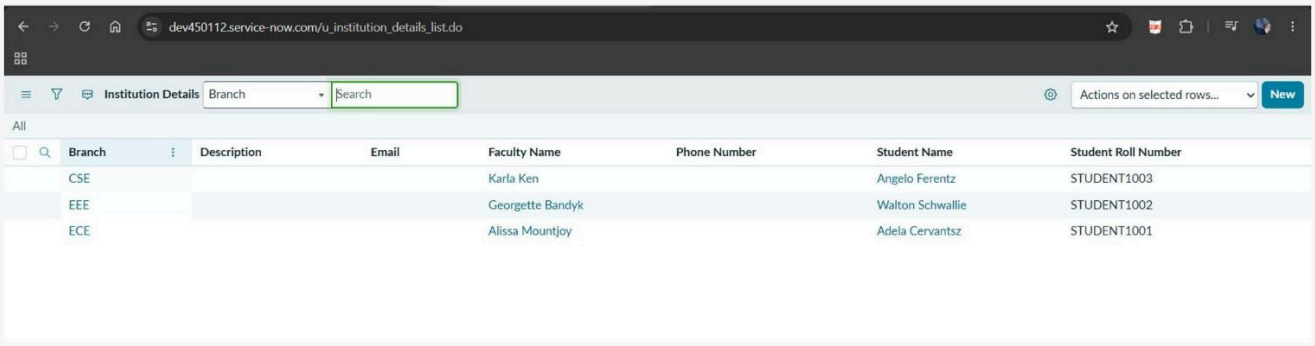
CUSTOM ROLES



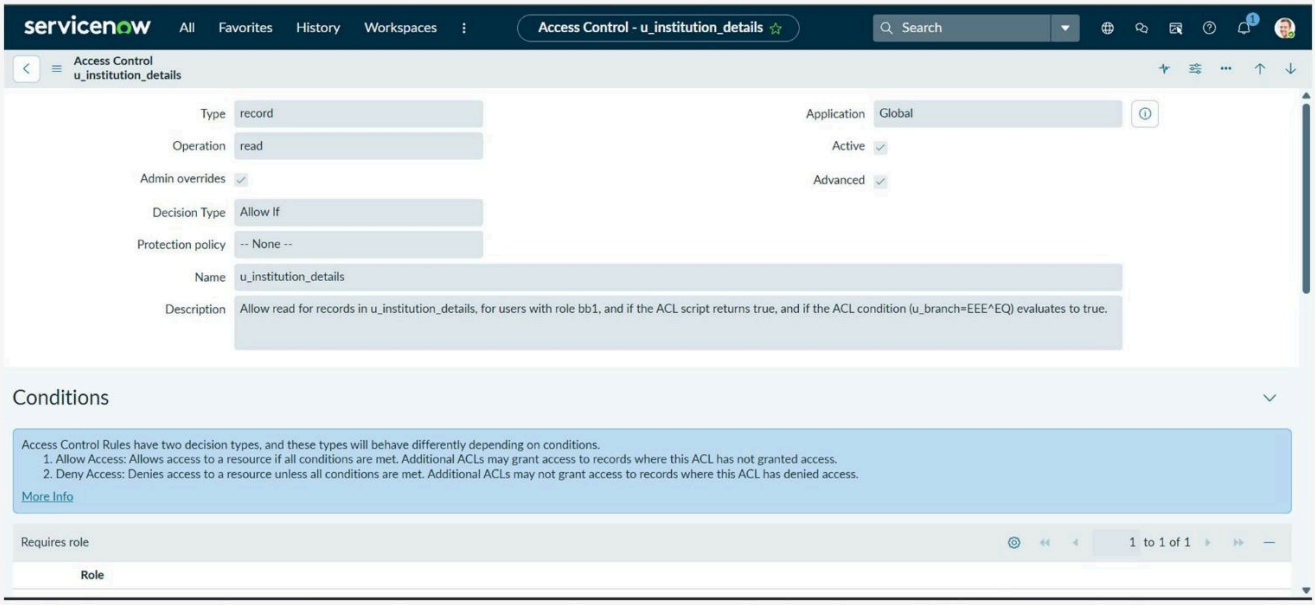
INSTITUTION TABLE DETAILS:



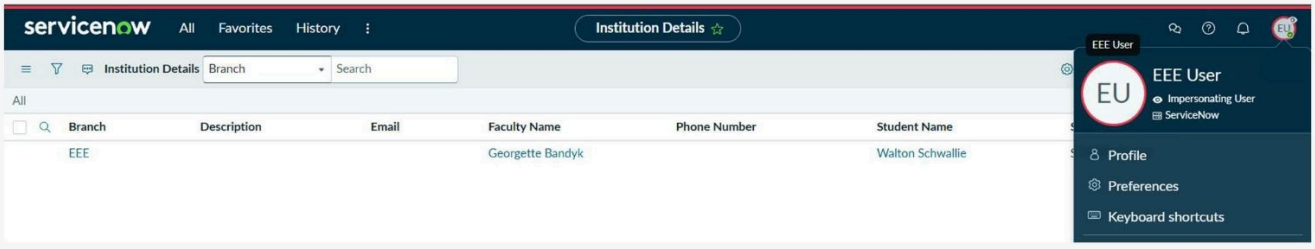
STUDENT RECORDS:



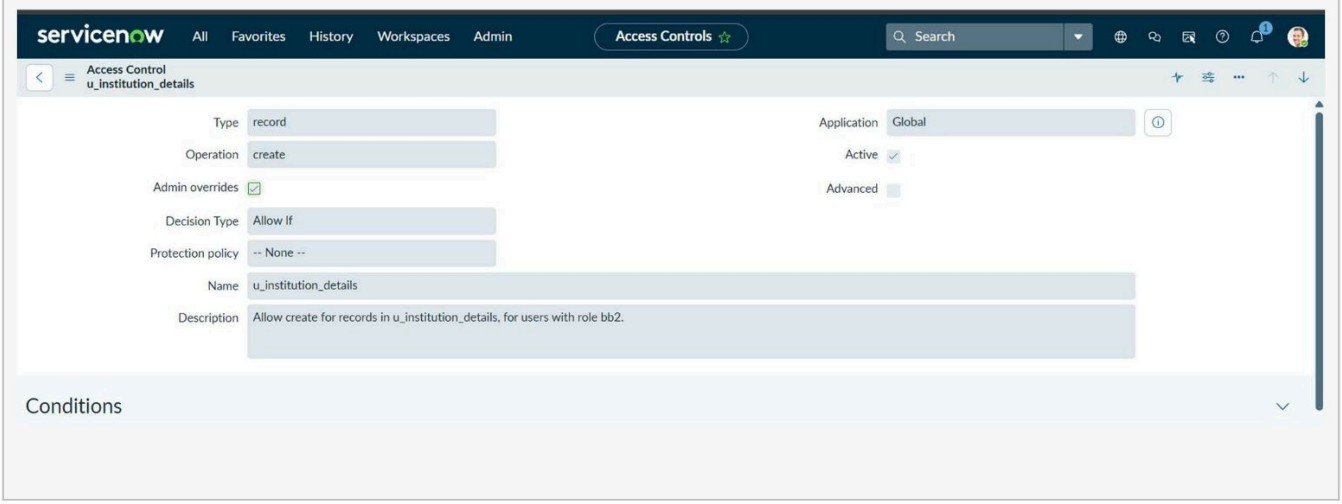
READ ACL:



EEE USER VERIFICATION:



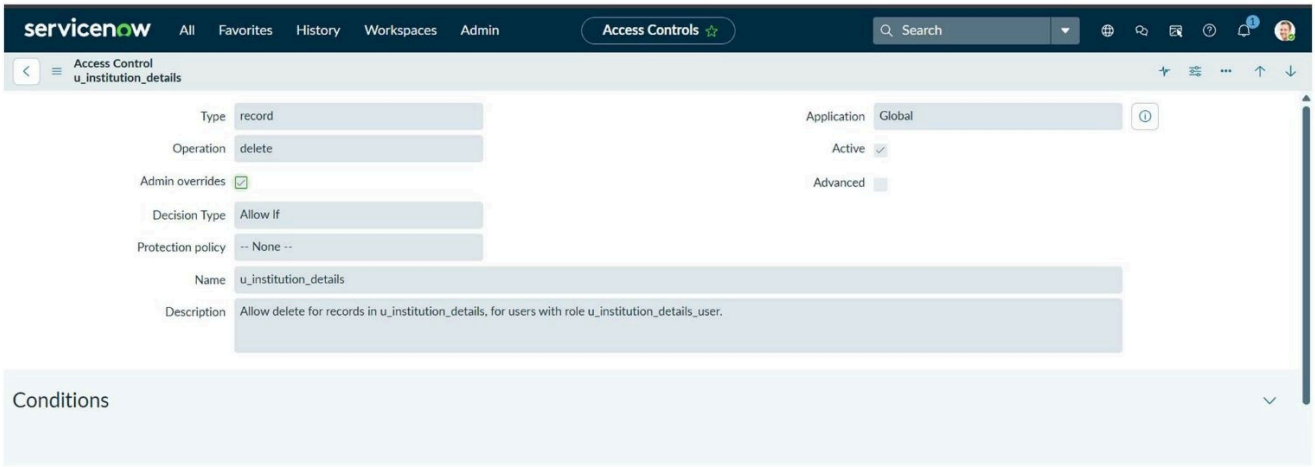
CREATE ACL:



WRITE ACL:



DELETE ACL:



8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides record-level access control through ServiceNow ACLs.
- Separates permissions by operation: Read, Create, Write and Delete.
- Allows the Read decision to consider both the user's role and a record field condition.
- Provides an explicit administrator access path in the supplied Read script.
- Supports verification using different combinations of custom roles.

Disadvantages / Limitations

- The supplied resource does not define a broader scalability or performance architecture.
- The project is implemented within a ServiceNow instance and does not specify a separate application database.
- The supplied resource does not specify external APIs, machine-learning services or separate file storage.
- The effectiveness of the demonstrated access policy depends on correct role assignment and ACL configuration.

9. CONCLUSION

The project demonstrates how Script-Controlled ACLs can enforce record-level security in ServiceNow. The Read ACL uses the bb1 role, a Branch = EEE condition and the supplied GlideSystem script, while separate ACLs control Create, Write and Delete operations using bb2, bb3 and bb4. The resulting workflow provides a clear way to verify access for EEE users, users without the required role, users with progressively more permissions, and administrators.

10. FUTURE SCOPE

The supplied project resource does not define a future-scope section. The following items are therefore presented as team-editable placeholders rather than claimed project requirements.

- Extend the access policy to additional departments or branches.
- Add more granular conditions for other record fields.
- Define measurable performance and load-testing requirements.
- Add a formal deployment and version-control process for ACL configuration.
- Extend verification to additional ServiceNow interfaces if required.

11. APPENDIX

Source Code (if any)

The main custom logic is the advanced Read ACL script. Insert the final script used in the ServiceNow instance below or attach it as an appendix page.

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }
```

```
    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')) {
        return true;
    }

    // Deny access for all others
    return false;
})();
```